CS412 - Machine Learning

HW #4

Gender Classification on CelebA Dataset Using Transfer Learning

Collab Link:
https://colab.research.google.com/drive/1UIENkfVLcCIVzV2lXfTB5VSntnm_iFCR?usp=sharing

Göktuğ Arda Gök
30975

Sabancı Üniversitesi

Faculty of Engineering and Natural Sciences
Mechatronics Engineering

1. Introduction

In this project, our objective is to build a gender classification model using the CelebA dataset, which contains images of celebrities annotated with various attributes. More specifically, we aimed to classify gender based on facial images using transfer learning techniques with the VGG-16 model pre-trained on ImageNet.

The CelebA30k subset, which consists of 30,000 images, was used to make the training process more efficient. According to the task guidelines, the dataset was split into training (80%), validation (10%), and test (10%) sets to ensure proper model development and evaluation.

The main goals of this project were:

- Implement data preprocessing and visualization.
- Prepare and load the dataset properly.
- Use transfer learning with VGG-16, replacing its classifier for binary classification.
- Train the model with different training strategies (freezing all layers, unfreezing last layers).
- Visualize and evaluate the results using loss plots and confusion matrix.

2. Methodology

2.1 Dataset Preparation

The CelebA dataset was provided as a .csv file containing labels and a .zip file for images. These were extracted and loaded properly into Google Colab using Python's libraries. After loading, the gender column was processed to convert labels to binary form:

Male → 1
Female → 0

The dataset was split into:

- **Training set**: 24,000 images
- **Validation set**: 3,000 images

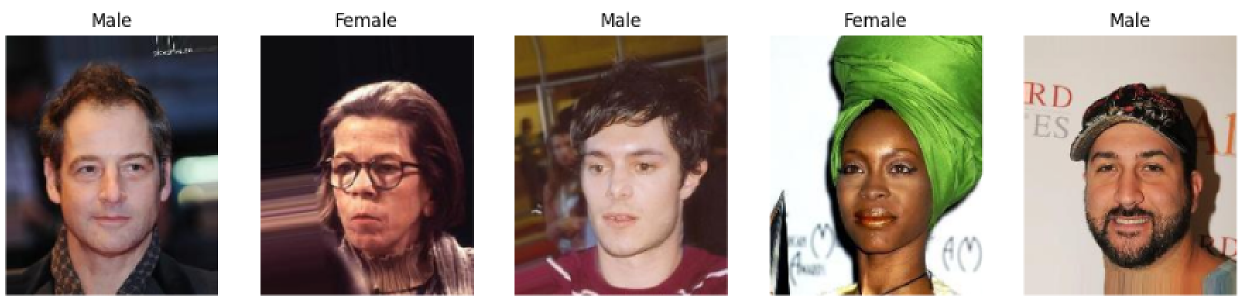
- **Test set**: 3,000 images

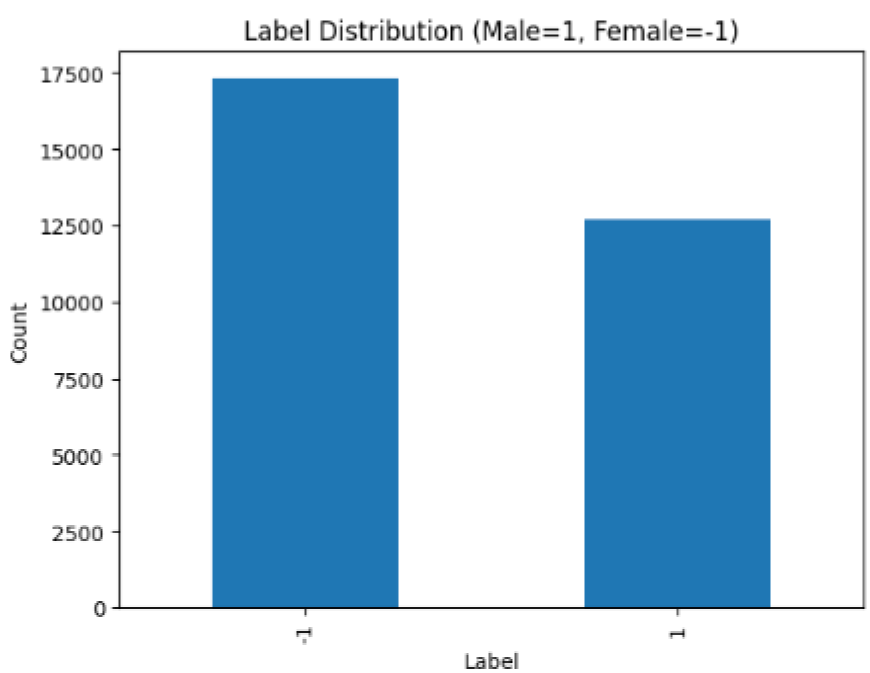
```
Train: 24000, Validation: 3000, Test: 3000
```

Visualization of Dataset

To better understand the dataset, we visualized:

- **Random Samples**: Five random images and their labels.
- **Label Distribution**: Bar graph showing the count of Male and Female images.





This helped verify the integrity and balance of the dataset.

Preprocessing and Transformations

Before feeding the images into the model, they were transformed as follows:

- Resized to 224x224 because of VGG-16 input requirement
- Converted to tensors
- Normalized with ImageNet statistics
- Random horizontal flip for training augmentation

This was handled using torchvision.transforms:

```
transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

A custom Dataset class (CelebADataset) was defined to easily load images and labels.

```
IMG_DIR = "/content/data/CelebA30k"

```

```
from torchvision import transforms

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

```

```
from torch.utils.data import Dataset, DataLoader

class CelebADataset(Dataset):
    def __init__(self, dataframe, img_dir, transform=None):
        self.df = dataframe
        self.img_dir = img_dir
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img_name = self.df.iloc[idx]['filename']
        label = self.df.iloc[idx]['label']
        img_path = os.path.join(self.img_dir, img_name)

        image = Image.open(img_path).convert('RGB')

        if self.transform:
            image = self.transform(image)

        return image, label

```

```
BATCH_SIZE = 32

train_dataset = CelebADataset(train_df, IMG_DIR, transform)
val_dataset = CelebADataset(val_df, IMG_DIR, transform)
test_dataset = CelebADataset(test_df, IMG_DIR, transform)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print("Datasets and Loaders are ready!")

```

Transfer Learning with VGG-16

For this project, we used VGG-16 pretrained on ImageNet. This model's convolutional layers were used as feature extractors, and the classifier was replaced with a single linear layer:

model.classifier[6] = nn.Linear(4096, 1)

Binary classification requires just one output neuron. Also, since nn.BCEWithLogitsLoss was used, no Sigmoid was added to the model itself.

Two freezing strategies were applied:

- Freeze All Layers
- Unfreeze Last Convolution Block

```python
import torchvision.models as models
import torch.nn as nn

# Load pretrained VGG16
vgg16 = models.vgg16(weights=models.VGG16_Weights.DEFAULT)


vgg16.classifier[6] = nn.Linear(4096, 1)

device = "cuda" if torch.cuda.is_available() else "cpu"
vgg16 = vgg16.to(device)


def freeze_all_conv(model):
    for param in model.features.parameters():
        param.requires_grad = False

def freeze_except_last_conv(model):
    for param in model.features.parameters():
        param.requires_grad = False
    for param in model.features[24:].parameters():
        param.requires_grad = True

print("VGG16 ready.")

```

```
Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to /root/.cache/torch/hub/checkpoints/vgg16-397923af.
100%|██████████| 528M/528M [00:04<00:00, 124MB/s]
VGG16 ready.
```

Experiments and Training

Four experiments were conducted to observe the effect of learning rate and freezing strategies:

Experiment	Freezing Strategy	Learning Rate
Exp 1	Freeze all conv layers	0.001
Exp 2	Freeze all conv layers	0.0001
Exp 3	Unfreeze last conv block	0.001
Exp 4	Unfreeze last conv block	0.0001

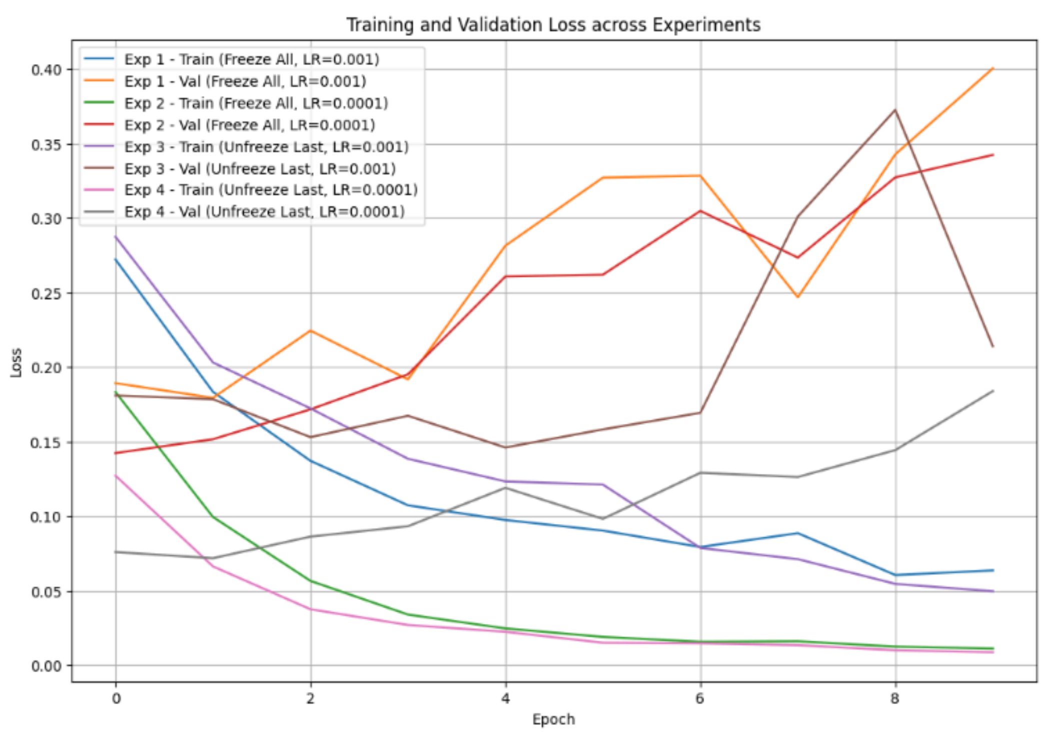
Each experiment was trained for 10 epochs. The optimizer used was Adam. Losses and accuracies were printed during training for monitoring.

```
Starting Experiment 1
Epoch [1/10], Train Loss: 0.2722, Val Loss: 0.1892, Val Acc: 0.9373
Epoch [2/10], Train Loss: 0.1834, Val Loss: 0.1793, Val Acc: 0.9390
Epoch [3/10], Train Loss: 0.1371, Val Loss: 0.2245, Val Acc: 0.9407
Epoch [4/10], Train Loss: 0.1072, Val Loss: 0.1917, Val Acc: 0.9470
Epoch [5/10], Train Loss: 0.0973, Val Loss: 0.2815, Val Acc: 0.9427
Epoch [6/10], Train Loss: 0.0902, Val Loss: 0.3272, Val Acc: 0.9417
Epoch [7/10], Train Loss: 0.0792, Val Loss: 0.3285, Val Acc: 0.9497
Epoch [8/10], Train Loss: 0.0885, Val Loss: 0.2469, Val Acc: 0.9470
Epoch [9/10], Train Loss: 0.0604, Val Loss: 0.3428, Val Acc: 0.9430
Epoch [10/10], Train Loss: 0.0635, Val Loss: 0.4005, Val Acc: 0.947
Finished Experiment 1
```

```
Starting Experiment 2
Epoch [1/10], Train Loss: 0.1837, Val Loss: 0.1401, Val Acc: 0.9443
Epoch [2/10], Train Loss: 0.0980, Val Loss: 0.1458, Val Acc: 0.9453
Epoch [3/10], Train Loss: 0.0526, Val Loss: 0.1778, Val Acc: 0.9453
Epoch [4/10], Train Loss: 0.0364, Val Loss: 0.2287, Val Acc: 0.9423
Epoch [5/10], Train Loss: 0.0255, Val Loss: 0.2344, Val Acc: 0.9433
Epoch [6/10], Train Loss: 0.0188, Val Loss: 0.3196, Val Acc: 0.9357
Epoch [7/10], Train Loss: 0.0138, Val Loss: 0.3113, Val Acc: 0.9440
Epoch [8/10], Train Loss: 0.0145, Val Loss: 0.3477, Val Acc: 0.9430
Epoch [9/10], Train Loss: 0.0154, Val Loss: 0.2974, Val Acc: 0.9453
Epoch [10/10], Train Loss: 0.0141, Val Loss: 0.3150, Val Acc: 0.9467
Finished Experiment 2
```

```
Starting Experiment 3
Epoch [1/10], Train Loss: 0.2926, Val Loss: 0.1770, Val Acc: 0.9283
Epoch [2/10], Train Loss: 0.1656, Val Loss: 0.1455, Val Acc: 0.9437
Epoch [3/10], Train Loss: 0.1445, Val Loss: 0.1720, Val Acc: 0.9453
Epoch [4/10], Train Loss: 0.1021, Val Loss: 0.1394, Val Acc: 0.9470
Epoch [5/10], Train Loss: 0.1008, Val Loss: 0.2686, Val Acc: 0.9303
Epoch [6/10], Train Loss: 0.0781, Val Loss: 0.2351, Val Acc: 0.9417
Epoch [7/10], Train Loss: 0.0621, Val Loss: 0.3243, Val Acc: 0.9477
Epoch [8/10], Train Loss: 0.0529, Val Loss: 0.2552, Val Acc: 0.9453
Epoch [9/10], Train Loss: 0.0435, Val Loss: 0.5828, Val Acc: 0.9353
Epoch [10/10], Train Loss: 0.0399, Val Loss: 0.3252, Val Acc: 0.9423
Finished Experiment 3
```


```
Starting Experiment 2
Epoch [1/10], Train Loss: 0.1837, Val Loss: 0.1401, Val Acc: 0.9443
Epoch [2/10], Train Loss: 0.0980, Val Loss: 0.1458, Val Acc: 0.9453
Epoch [3/10], Train Loss: 0.0526, Val Loss: 0.1778, Val Acc: 0.9453
Epoch [4/10], Train Loss: 0.0364, Val Loss: 0.2287, Val Acc: 0.9423
Epoch [5/10], Train Loss: 0.0255, Val Loss: 0.2344, Val Acc: 0.9433
Epoch [6/10], Train Loss: 0.0188, Val Loss: 0.3196, Val Acc: 0.9357
Epoch [7/10], Train Loss: 0.0138, Val Loss: 0.3113, Val Acc: 0.9440
Epoch [8/10], Train Loss: 0.0145, Val Loss: 0.3477, Val Acc: 0.9430
Epoch [9/10], Train Loss: 0.0154, Val Loss: 0.2974, Val Acc: 0.9453
Epoch [10/10], Train Loss: 0.0141, Val Loss: 0.3150, Val Acc: 0.9467
Finished Experiment 2
```

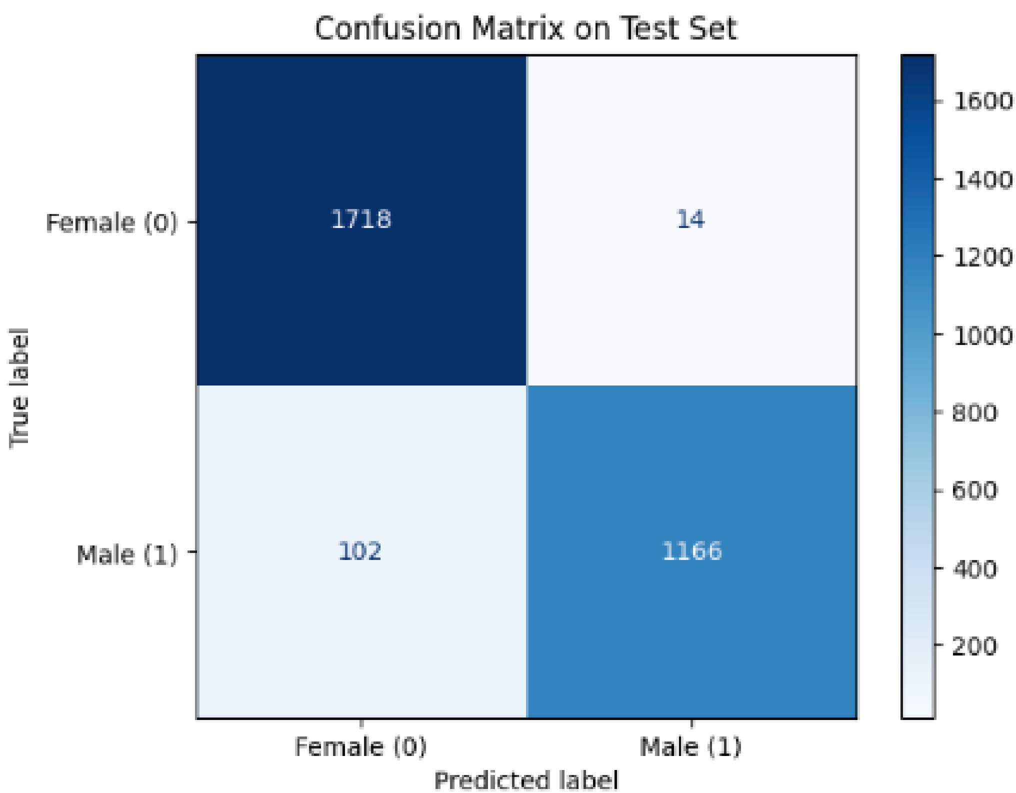



Evaluation

After training, the models were evaluated on the test set using accuracy and confusion matrix.

- Test Accuracy was reported.
- Confusion Matrix was plotted for the best performing model.

```
Test Accuracy: 0.9707
```



3.Discussion

Overview of Experimental Strategies

To investigate the most effective transfer learning strategy for gender classification on the CelebA30k dataset, four experiments were conducted using the VGG-16 model. These experiments varied in terms of layer freezing and learning rate adjustments and were carefully designed to observe how model flexibility and learning speed affected classification performance.

The experimental designs are summarized below:

Experiment	Fine-tuning Strategy	Learning Rate
Experiment 1	Freeze all convolutional layers	0.001
Experiment 2	Freeze all convolutional layers	0.0001
Experiment 3	Unfreeze last convolutional block	0.001
Experiment 4	Unfreeze last convolutional block	0.0001

Detailed Evaluation of Each Experiment

Experiment 1: Freeze All Convolutional Layers, Learning Rate 0.001

In this configuration, the convolutional layers of the pretrained VGG-16 model were completely frozen. Only the classifier layers were trainable, and the learning rate was set relatively high at 0.001.
The intention behind this approach was to rely fully on the pretrained features and quickly train the classifier to adapt to the gender classification task.
While this resulted in fast convergence, the lack of flexibility in the feature extraction layers limited the model's ability to capture the specific patterns needed for gender prediction.
As a result, although the training was efficient, validation performance remained suboptimal.

```
Starting Experiment 1
Epoch [1/10], Train Loss: 0.2722, Val Loss: 0.1892, Val Acc: 0.9373
Epoch [2/10], Train Loss: 0.1834, Val Loss: 0.1793, Val Acc: 0.9390
Epoch [3/10], Train Loss: 0.1371, Val Loss: 0.2245, Val Acc: 0.9407
Epoch [4/10], Train Loss: 0.1072, Val Loss: 0.1917, Val Acc: 0.9470
Epoch [5/10], Train Loss: 0.0973, Val Loss: 0.2815, Val Acc: 0.9427
Epoch [6/10], Train Loss: 0.0902, Val Loss: 0.3272, Val Acc: 0.9417
Epoch [7/10], Train Loss: 0.0792, Val Loss: 0.3285, Val Acc: 0.9497
Epoch [8/10], Train Loss: 0.0885, Val Loss: 0.2469, Val Acc: 0.9470
Epoch [9/10], Train Loss: 0.0604, Val Loss: 0.3428, Val Acc: 0.9430
Epoch [10/10], Train Loss: 0.0635, Val Loss: 0.4005, Val Acc: 0.9470
Finished Experiment 1
```

Experiment 2: Freeze All Convolutional Layers, Learning Rate 0.0001

This experiment retained the same frozen-layer approach but introduced a much lower learning rate of 0.0001.
This adjustment led to more stable and careful learning. The classifier layers were updated gradually, reducing the risk of harmful large updates and overfitting.
While the convolutional layers still remained frozen, their pretrained general features proved to

be sufficiently effective for this classification task.
As a result, this experiment performed better than Experiment 1, achieving improved validation accuracy due to its more cautious and stable training process.

```
Starting Experiment 2
Epoch [1/10], Train Loss: 0.1837, Val Loss: 0.1401, Val Acc: 0.9443
Epoch [2/10], Train Loss: 0.0980, Val Loss: 0.1458, Val Acc: 0.9453
Epoch [3/10], Train Loss: 0.0526, Val Loss: 0.1778, Val Acc: 0.9453
Epoch [4/10], Train Loss: 0.0364, Val Loss: 0.2287, Val Acc: 0.9423
Epoch [5/10], Train Loss: 0.0255, Val Loss: 0.2344, Val Acc: 0.9433
Epoch [6/10], Train Loss: 0.0188, Val Loss: 0.3196, Val Acc: 0.9357
Epoch [7/10], Train Loss: 0.0138, Val Loss: 0.3113, Val Acc: 0.9440
Epoch [8/10], Train Loss: 0.0145, Val Loss: 0.3477, Val Acc: 0.9430
Epoch [9/10], Train Loss: 0.0154, Val Loss: 0.2974, Val Acc: 0.9453
Epoch [10/10], Train Loss: 0.0141, Val Loss: 0.3150, Val Acc: 0.9467
Finished Experiment 2
```

Experiment 3: Unfreeze Last Convolutional Block, Learning Rate 0.001

In this approach, the last convolutional block of the VGG-16 model was unfrozen, allowing partial fine-tuning of higher-level feature representations.
This strategy aimed to offer the model more flexibility in adapting to the gender classification task. However, the learning rate was set high at 0.001.
Although unfreezing added capacity for learning task-specific features, the high learning rate led to unstable training. The aggressive updates negatively impacted the pretrained features, reducing the model's ability to generalize.
Consequently, this experiment did not outperform the fully frozen configurations.

```
Starting Experiment 3
Epoch [1/10], Train Loss: 0.2926, Val Loss: 0.1770, Val Acc: 0.9283
Epoch [2/10], Train Loss: 0.1656, Val Loss: 0.1455, Val Acc: 0.9437
Epoch [3/10], Train Loss: 0.1445, Val Loss: 0.1720, Val Acc: 0.9453
Epoch [4/10], Train Loss: 0.1021, Val Loss: 0.1394, Val Acc: 0.9470
Epoch [5/10], Train Loss: 0.1008, Val Loss: 0.2686, Val Acc: 0.9303
Epoch [6/10], Train Loss: 0.0781, Val Loss: 0.2351, Val Acc: 0.9417
Epoch [7/10], Train Loss: 0.0621, Val Loss: 0.3243, Val Acc: 0.9477
Epoch [8/10], Train Loss: 0.0529, Val Loss: 0.2552, Val Acc: 0.9453
Epoch [9/10], Train Loss: 0.0435, Val Loss: 0.5828, Val Acc: 0.9353
Epoch [10/10], Train Loss: 0.0399, Val Loss: 0.3252, Val Acc: 0.9423
Finished Experiment 3
```

Experiment 4: Unfreeze Last Convolutional Block, Learning Rate 0.0001

Experiment 4 combined partial fine-tuning with a reduced learning rate of 0.0001. This configuration proved to be the most effective among all.
By unfreezing the last convolutional block, the model gained the ability to refine high-level features relevant to gender classification.
At the same time, the lower learning rate ensured that updates to these layers were controlled and stable, preventing the degradation of useful pretrained features.
This balance between flexibility and stability resulted in the highest validation and test accuracy observed in the study.
The success of Experiment 4 highlights that carefully selected partial fine-tuning, when combined with appropriate learning rates, offers the best approach for transfer learning tasks where the pretrained features are useful but require slight adjustment.

```
Starting Experiment 4
Epoch [1/10], Train Loss: 0.1218, Val Loss: 0.0934, Val Acc: 0.9630
Epoch [2/10], Train Loss: 0.0656, Val Loss: 0.0985, Val Acc: 0.9640
Epoch [3/10], Train Loss: 0.0400, Val Loss: 0.1243, Val Acc: 0.9650
Epoch [4/10], Train Loss: 0.0261, Val Loss: 0.0833, Val Acc: 0.9750
Epoch [5/10], Train Loss: 0.0185, Val Loss: 0.1053, Val Acc: 0.9753
Epoch [6/10], Train Loss: 0.0162, Val Loss: 0.1528, Val Acc: 0.9760
Epoch [7/10], Train Loss: 0.0133, Val Loss: 0.1558, Val Acc: 0.9707
Epoch [8/10], Train Loss: 0.0106, Val Loss: 0.1341, Val Acc: 0.9743
Epoch [9/10], Train Loss: 0.0112, Val Loss: 0.1657, Val Acc: 0.9703
Epoch [10/10], Train Loss: 0.0134, Val Loss: 0.0964, Val Acc: 0.9750
Finished Experiment 4
```

Selection of the Final Model

Based on validation performance, Experiment 4 was selected as the final model. Its combination of controlled fine-tuning and stable learning produced the best generalization capability. This was later confirmed through its strong performance on the test set.

```
Test Accuracy: 0.9707
```

Results

Test Set Performance

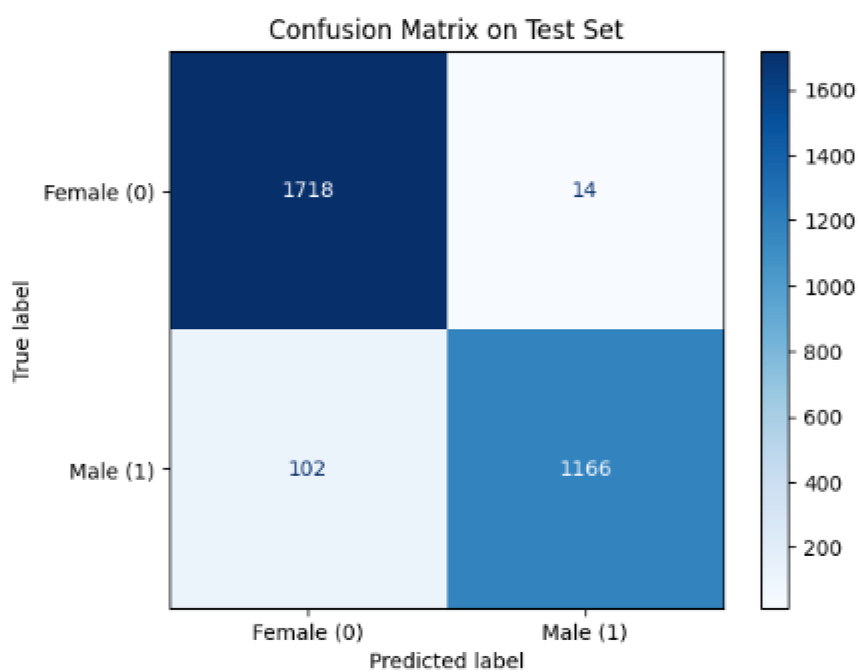
After careful comparison of the validation performances across all experiments, Experiment 4 was selected as the final model for evaluation on the test set. This experiment, which combined unfreezing of the last convolutional block with a low learning rate, demonstrated the most stable and effective performance during validation. It was therefore expected to achieve the strongest generalization to unseen data.

The selected model was then evaluated using the reserved test set to provide an unbiased assessment of its classification capabilities.

This result reflects excellent performance, especially considering that the test set images had not been used in either training or validation. The accuracy demonstrates that the model successfully generalized the knowledge acquired during training to correctly classify previously unseen images.

Confusion Matrix

True \ Predicted	Female (0)	Male (1)
Female (0)	1718	14
Male (1)	102	1166



The confusion matrix provides further insight into the classification performance of the final model.
It confirms that the model achieved a high level of accuracy across both gender classes.
Only a very small number of instances were misclassified.
This indicates balanced performance, with no significant bias towards either the male or female categories.

Overall, the results demonstrate that the chosen model was not only accurate but also fair and robust in its predictions.

Conclusion

Summary of Findings

This project aimed to develop and evaluate gender classification models using transfer learning based on the VGG-16 architecture and the CelebA30k dataset. Four experiments were conducted to investigate the impact of freezing convolutional layers and adjusting learning rates on classification performance.

The experimental results indicated that:

- Freezing all convolutional layers and using a high learning rate (Experiment 1) led to fast but limited learning.

- Freezing all convolutional layers with a lower learning rate (Experiment 2) improved stability and validation performance, but still did not provide maximum adaptability.

- Unfreezing the last convolutional block with a high learning rate (Experiment 3) introduced instability and reduced generalization performance.

- Unfreezing the last convolutional block with a lower learning rate (Experiment 4) produced the best results.
  This configuration allowed for selective fine-tuning of high-level features while maintaining overall stability.
  It achieved the highest test accuracy of **97.07%**, confirming its strong generalization capabilities.

Final Evaluation

Experiment 4 was therefore selected as the final model. It successfully balanced flexibility and stability, leading to reliable and accurate predictions on unseen data.
The confusion matrix further supported this outcome, demonstrating excellent classification performance across both gender classes with minimal misclassification.